

NLP Course Template

Khakhaleva Tatyana

May 2026

Abstract

This project is dedicated to the problem of chat bots moderation. A commercial company gave my team a large dataset of multiple chat bots logs in order to develop a solution for automatic detection of illegal drug trade. The binary classification problem was solved in several ways: rule-based method, self-hosted LLMs + in context learning. A comparative analysis of the results of these approaches was carried out. Please provide a link to your project code right here: <https://github.com/tatyanalaugh/NLP-ODS-Project>.

1 Introduction

Social media have long ago become not only a place for communication but also a platform where different businesses sell products and services and find their clients. Therefore it's really important that representation of the company in messengers creates a good impression and doesn't harm the company's reputation. The existence of chat bots which were made with templates of a legal company to promote and sell drugs is absolutely not acceptable.

This problem was observed in various articles on the material of Western social media such as Instagram and Reddit. There were published a couple of datasets which represent different kinds of illegal content for its detection with Machine Learning methods.

Nevertheless, we haven't found a well-known solution for the Russian domain. There aren't any big datasets which can cover Russian lexical specific of illegal drug trade quite enough. Moreover, it seems that none of the articles concerned with the problem offers an approved illegal content detection framework for the Russian language.

In this study we present several simple solutions which can be implemented on business data and don't require much technical capacity and knowledge to be built in.

1.1 Team

This project was prepared by Khakhaleva Tatyana, Odnoshivkina Victoria and Mamochkina Ekaterina. Approximate distribution of responsibilities is considered below.

Khakhaleva Tatyana: raw dataset processing, dataset description, existing approaches survey, dictionary enlarging, DuckDB pipeline for quick searching through dataset, LLM estimation pipeline, metrics, report preparation.

Odnoshivkina Victoria: raw dataset processing, dictionary creation, metrics.

Mamochkina Ekaterina: raw dataset processing, existing approaches survey, in context learning, metrics.

2 Related Work

Possible solutions for illicit content detection can be separated into three main groups: keyword-based method; ML and DL methods, e.g. Word2Vec, TF-IDF, LSTM, ULMFit, BERT, RoBERTa; LLM prompting. There are also differences in collecting and processing data for further work. In this section we are going to observe previous art in obtaining and preparing the data along with building classification kits.

2.1 Data Acquisition and Preparation

The literature reveals two dominant sourcing strategies: crawling dark web domains and scraping mainstream social media or messaging platforms.

On the dark web side, the CoDA corpus comprises 10,000 documents crawled from real `.onion` websites on the Tor network and manually annotated across ten illicit categories. In the study of Prado-Sánchez researcher team near-duplicate documents were removed using cosine similarity over TF-IDF vectors with a 0.95 threshold [3]. A complementary dataset of 113,995 instances was built by merging the Agora and Duta10k datasets with onion sites and surface-web pages collected through the HTTrack tool, text was extracted from HTML and preprocessed through tokenization, stop-word removal with NLTK, and lemmatization [1].

On the social media side, Li et al. scraped 12,857 Instagram posts via controlled-substance hashtag searches and cleaned them by removing duplicates, hyperlinks, special characters, and stop words. Ground truth was established through manual binary annotation, achieving an intercoder reliability of $\kappa = 0.98$ [2]. Shah et al. gathered 43,607 Instagram comments using more than 1,000 drug-related hashtags and isolated relevant content through iterative unsupervised clustering before manual annotation [5]. Sonawane et al. collected Telegram messages and standardized them through text cleaning, language detection and translation, emoji tagging, and duplicate removal [6]. The JEDIS framework avoided manual labeling by applying distant supervision to the Reddit Drug and Silk Road Forum corpora, automatically labeling data using a small list of DEA-derived seed terms [7].

2.2 Keyword- and Lexicon-Based Methods

The earliest and simplest approaches rely on matching text against predefined lexicons. Sonawane et al. detected suspicious Telegram messages by matching drug names and slang and by applying contextual scoring, in which cues such as numbers followed by “mg” or “g” or currency symbols adjacent to drug names increased a message’s suspiciousness score [6]. Statistical term-weighting schemes have been widely applied to darknet content, including Bag-of-Words and TF-IDF representations. Al Nabki et al. combined TF-IDF with logistic regression to reach 96.6% accuracy. However, Choi et al. demonstrated that such statistical methods fall behind in classification because they cannot comprehend the semantic meaning of human-generated text [1]. The fundamental fragility of lexicon-based moderation is articulated most clearly by the JEDIS authors, who note that banlists are trivially evaded through word substitutions and misspellings (e.g., *cokaine*, *c0caine*) and that euphemistic terms with benign meanings (e.g., *pot*, *crack*, *speed*) generate false positives. Empirically, word-based baselines such as Word2Vec, CantReader, and the model of Zhu et al. attain high precision but suffer from low recall, missing many euphemisms that adversaries can exploit [7].

2.3 Machine Learning and Deep Learning Methods

To overcome the semantic limitations of lexicon matching, subsequent work adopted trainable models ranging from classical classifiers to transformer architectures. Li et al. employed a recurrent neural network with a long short-term memory (LSTM) unit to learn the textual patterns of drug-dealing posts, comparing it against decision tree, random forest, and support vector machine baselines; the deep learning model achieved the best F1 score of roughly 0.95 [2]. Rupa and Gangopadhyay extended this idea to a multimodal setting, fusing image features from a YOLOv3-based drug-type detector with text features at the feature level and feeding them into a deep learning model [4]. Shah et al. instead used the unsupervised Biterm Topic Model to summarize short Instagram comments into coherent topics suitable for signal extraction [5]. Within the transformer family, Cascavilla et al. compared ULMFit, BERT, RoBERTa finding BERT to be the strongest model with 96.08% accuracy on general illicit-activity classes and 91.98% on drug subtypes, while ULMFit proved poorly scalable on the smaller drug subset [1]. JEDIS advanced this line further by combining a dellexicalized context-only BERT module with a word-attribute module trained through distant supervision, outperforming both word-based baselines and context-based baselines such as DarkBERT in F1 score [7]. But such supervised and fine-tuned transformer models require substantial labeled data and domain-specific retraining that constrain rapid deployment [3].

2.4 Large Language Model Prompting

The most recent direction replaces task-specific training with natural-language prompting of LLMs. Prado-Sánchez et al. evaluated eight commercial LLMs on the CoDA corpus under zero-shot conditions, with DeepSeek Chat, Grok, and Gemini 2.0 Flash leading at macro-F1 scores of 0.870, 0.868, and 0.861, respectively. These models aligned closely with human annotations, achieving weighted Cohen’s Kappa values above 0.86 for the top performers and a global Krippendorff’s Alpha of 0.871. Although task-specific supervised classifiers still slightly outperformed the best LLM, the zero-shot approach required no fine-tuning, which makes it attractive when labeled data are scarce [3]. A more critical observation comes from the JEDIS evaluation, whose experiments showed that GPT-4o-mini guided by detailed prompts achieved high recall and coverage but markedly low precision [7].

3 Model Description

In general, we have used two methods to solve the binary classification problem for drug-related content. The first approach is the rule-based method and the second approach is prompting of LLMs along with regulating parameters, i.e. choosing temperature, maximal tokens number, enabling or disabling reasoning mode.

3.1 Rule-based method

The simplest method to solve our problem is known as the rule-based method. Its essence lies in the fact that flagging illegal content is based on a certain list of suspicious vocabulary which can be collected in different ways. In our project we have fulfilled several strategies of building a dictionary.

First of all, we have defined a relevant sources pool. It consists of four sources:

1. Russian Illegal Drugs Reviews Sentiment Dataset [8]. These reviews were collected via web scraping of popular Russian darknet market Hydra. The dataset contains some interesting lexical units. Victoria has analyzed the content column and separated its vocabulary in a row of categories: *drugs*, *drug_trade*, *drug_effect*, *instruments*, *general*. The separation wasn’t applied on this solution level but turned out to be useful in LLMs prompting. Victoria has also prepared a short description of each term’s meaning.
2. Russian drug addicts slang [9]. This is one of multiple sites which offer lists of Russian drug addicts slang. It was used for getting above-mentioned descriptions.
3. Wiki Dictionary Russian drug addicts slang [10]. This is probably the largest list of drug-related vocabulary in the Russian language. Wiki Dictionary was used for enlarging our manual collected dictionary. I have written a Python script which separates a term from its synonym by a comma and synonymous terms from their meaning by em dash along with cleaning formatting. Meanings were added to the description column. Only unique words compared with the already existing dictionary were added into this dictionary.
4. Drug Seed Terms [11]. A list of English Terms offered in Song’s work for constructing the training dataset [7]. The EDA of our dataset has shown that messages also can contain English names of drugs, so we decided to add key English terms to our dictionary (Table 1).

Drug Seed Terms						
adderal	alprazolam	amphetamine	clonazepam	cocaine	concerta	daytrana
dilaudid	ecstasy	fentanyl	flunitrazepam	ghb	heroin	hydrocodone
hydromorphone	ketalar	ketamine	khat	klonopin	lorcet	lsd
marijuana	mdma	mescaline	methamphetamine	methylphenidate	molly	morphine
norco	opium	oxaydo	oxycodone	oxycontin	pcp	percocet
peyote	phencyclidine	promethazine	ritalin	rohypnol	roxicodone	steroids
suboxone	u-47700	vicodin	xanax			

Table 1: Drug seed terms (English) extracted from the DEA report [11] and added to the dictionary.

We have also considered adding some data from the CoDA dataset that was mentioned in the Related Work section. It has about 500 Russian documents. Unfortunately, usage of this dataset is restricted to research and/or academic purposes only, whereas we were making a solution for a commercial company. There is also an open-access dataset of inappropriate sensitive topics in the Russian language having drugs subsection [12, 13]. It could be used for further enlarging of our dictionary, but requires some more processing because alcohol is also considered as a drug there and not all its messages are actually discussing illegal subjects, some of them are common discussion about drugs and alcohol. At the current stage of project development we have decided to stop on our dictionary as long as it has shown an acceptable F1-score.

In the upshot, we have formed the CSV dictionary with columns: **term**, **normalized_term**, **language**, **category**, **description**, **example**, **regex_pattern**. The dictionary contains 840 rows.

The search through the dataset with the dictionary was fulfilled through SQL-queries using DuckDB Python package. You can find a more precise description of the pipeline in the Experiment Setup section.

3.2 Instruction-tuned LLMs

We have tested various LLMs on the binary classification task. LLMs were deployed on our local server using a 32 GB GPU to avoid possible leakage of sensitive customers data. We have explored following models:

- Qwen3.5-9B;
- Gemma-4-E4B-it;
- YandexGPT-5-Lite-8B-instruct;
- Ministral-3-8B-Instruct-2512;
- Phi-4-mini-instruct.

As it is obvious from the list above, we haven’t deployed heavy models with large numbers of parameters intentionally. Our aim was to develop an effective solution that won’t require much resources but will demonstrate the quality that is enough for lowering chat bots moderators workload.

3.2.1 Estimation Notebook

Figure 1 gives an overview of the end-to-end pipeline, and Figure 2 details the fault-tolerant control flow of a single classification request.

The pipeline is organised as a linear sequence of stages: (i) construction of a domain lexicon; (ii) preparation of model inputs from the raw corpus; (iii) assembly of the prompt; (iv) constrained LLM inference; and (v) evaluation against ground-truth labels. All stages share a single deterministic configuration so that a run can be reproduced exactly and resumed after interruption.

A curated dictionary of illicit-domain terms is supplied to the model as auxiliary knowledge. The lexicon is normalised before use. The resulting items are serialised to JSON. Two views of the lexicon are produced: a full view (**DRUG_TERMS**), which retains every category present in the source and a compact view (**DRUG_TERMS_SHORT**), restricted to the *drugs* category. Each prompt variant embeds one of these two views.

The evaluation corpus is stored in Parquet, where each row corresponds to one conversational turn under a `session_id`. For each turn a composite identifier of the form `session_id__turn_idx` is created, where `turn_idx` is the zero-based position of the turn within its session; this guarantees a globally unique key even though a single session contains several turns. The text passed to the model concatenates both sides of the turn as “Вопрос: {question}” / “Ответ: {answer}”. Each input item is thus a minimal JSON object containing only an `id` and a `text` field.

Every request is composed of a system message that establishes the moderation role and a user message produced by a prompt builder. Three prompt variants are evaluated.

1. A detailed specification that defines the positive class through several explicit conditions (direct mentions, slang and euphemisms, deliberately obfuscated spellings, transliteration, action context, and emoji signals), defines the negative class (homonyms in everyday usage, medical context without abuse, irony, unrelated chatter), states an explicit decision priority favouring context over isolated tokens, and embeds the full lexicon `DRUG_TERMS`.
2. A shorter formulation that relies on the compact lexicon `DRUG_TERMS_SHORT` and a reduced rule set.
3. Extended first prompt with additional positive-class triggers, including entities of the drug trade, obfuscated channel and bot handles, suspicious abbreviations and isolated Latin characters, cryptocurrency references, and double-meaning phrasing.

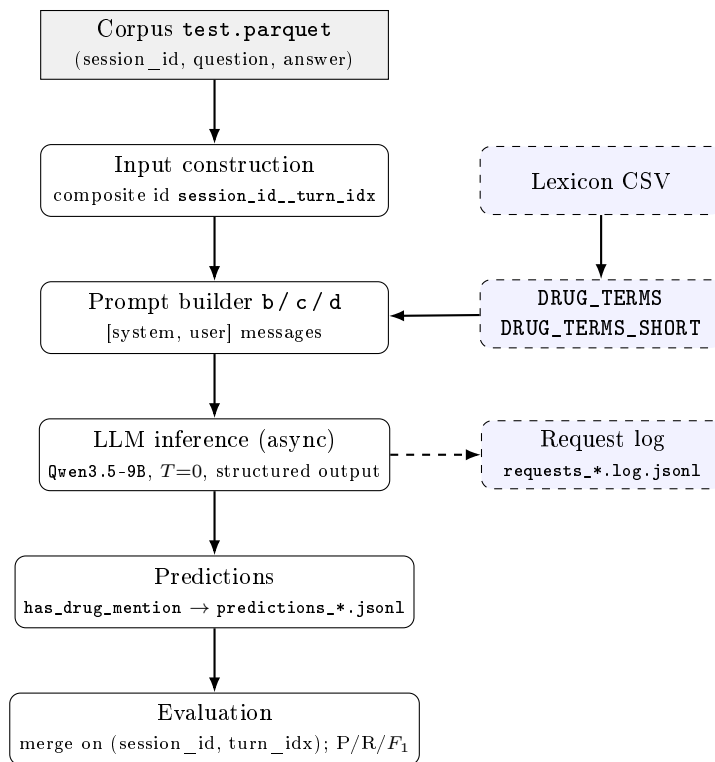


Figure 1: End-to-end LLM moderation pipeline. The curated lexicon is embedded into the prompt; every inference call is logged, and predictions are evaluated against ground truth on the composite turn key.

All variants specify the same input and output contracts, instruct the model to use only the `text` field, and prohibit any output other than the target JSON object.

To eliminate format drift, the model response is constrained by a minimal schema with a single Boolean field:

`DrugMentionClassification = { has_drug_mention : bool }.`

This schema is passed as the response format, so decoding is constrained to a valid instance of the schema.

Inference is performed against an OpenAI-compatible serving endpoint through an asynchronous client. Requests are issued concurrently up to a bound of 32 simultaneous in-flight calls.

Each request is wrapped in a retry loop of up to five attempts (Figure 2). Errors are partitioned into a taxonomy: connection errors, rate-limit errors, timeouts, and server-side status errors are treated as transient and retried, whereas client-side status errors are treated as fatal and abort the request without retry. A response that cannot be parsed into the schema is also retried. Every attempt is recorded to a line-delimited JSON log under an asynchronous lock, capturing the timestamp, identifier, prompt variant, attempt number, elapsed time, a SHA-256 hash of the prompt, the raw and parsed responses, and the resulting status. Each final prediction is appended to a predictions file. Before a run, the set of already-completed identifiers is loaded from the predictions file so that only outstanding items are processed, making the procedure resumable after interruption.

For evaluation, predictions are joined with the ground-truth corpus on the composite key (`session_id`, `turn_idx`). Duplicate identifiers arising from repeated runs are resolved by keeping the most recent prediction, and items for which the model failed to return a verdict are excluded from the metric computation and reported separately. The reference label is binarised as (`message_label = illegal`) and compared against the model’s `has_drug_mention` verdict.

A dedicated error analysis separates FN from FP and exports them for manual inspection.

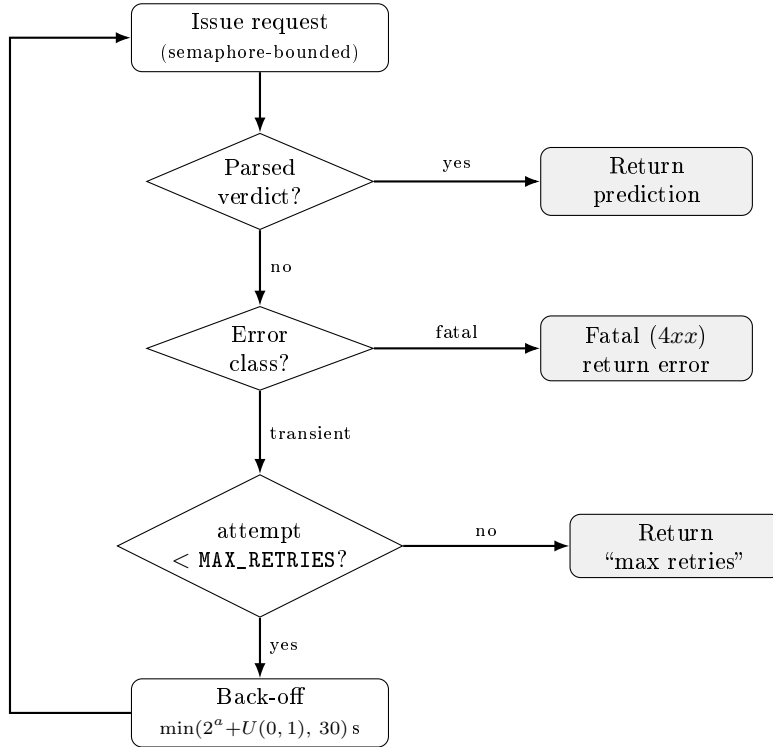


Figure 2: Fault-tolerant control flow of a single classification request. Transient failures (connection, rate-limit, timeout, 5xx, or parse failures) are retried with exponential back-off and jitter; client-side (4xx) errors are fatal.

3.2.2 In context learning

In-context learning (ICL) is the ability of LLMs to learn a task from a handful of labeled demonstrations supplied directly in the prompt, without any update to the model’s parameters. Formally, the model is given a task description, a set of labeled demonstrations, and a query, and predicts the answer by conditioning on this concatenated context (Figure 3). Because ICL is highly sensitive to which demonstrations are chosen,

demonstration selection is a key design lever: balancing relevance, class coverage, and prompt length directly affects both accuracy and cost.

In this project each prompt combines a task description, a few-shot block of labeled examples drawn from the training set, and an optional domain glossary of slang terms. To study the effect of demonstration choice, we compare three sampling strategies (random, 1:1 class-balanced, and 2:1 legal/illegal) with and without the glossary.

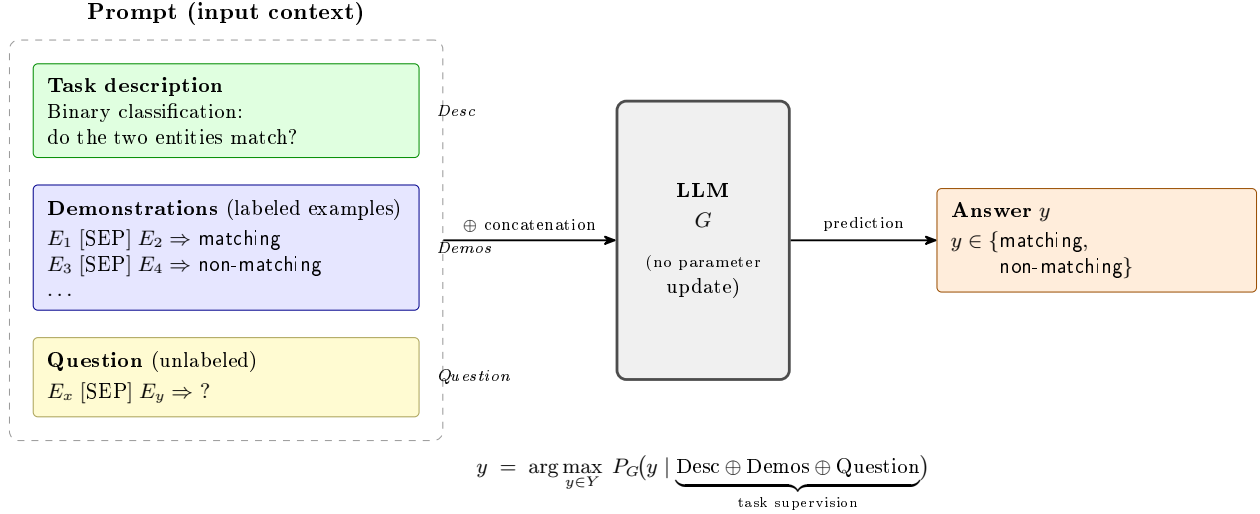


Figure 3: In-context learning scheme. The prompt concatenates a task description, labeled demonstrations, and an unlabeled question; the frozen LLM G predicts the answer y without any parameter update.

4 Dataset

At the beginning of the project we received a raw dataset of chat bots logs from the analytics team. It consisted of 64 columns, some columns included JSON files that duplicated information from the other columns. Therefore we had to decide which columns should stay in the final dataset. After some discussions and questioning the analytics team how several columns were formed we have finalized the dataset. The final dataset is a PARQUET file containing 10 columns and 7 816 579 rows. Columns are represented in Table 2.

Column	Data type	Description
timestamp	int64	Message/event time with millisecond precision. Example: 1544560105439.
account_id	large_string	The numeric ID of the account that owns the bot. Example: 175.
project_short_name	large_string	Project string identifier. Example: test-175-DAM. They are generated according to the following scheme: bot name (for example, apartamentos_espana) – account_id (2587450243) – letter code (9XC). These three parts are separated by a hyphen.
channel_type	large_string	The type of channel where the bot can be published. Example: chatapi.
question	large_string	Client message. Can be empty if an event was sent or the table row contains an operator message.
nlp_class	large_string	The state that the request entered after the matcher. May not be specified. Example: /3.
answer	large_string	The bot or operator's response. May be empty if the table row contains data about the client's incoming message.
session_id	large_string	String session identifier. Example: chatapi-175-test-175-DAM-2708-test.47b21ba7-4f94-4408-8549-5cd3ebcd42a3.
message_label	large_string	Marking specific messages (legal/illegal).
from_illegal_account	int64	Binary flag of illegal accounts.

Table 2: Columns of the final dataset.

Each row of the dataset corresponds to one conversational turn with a **question** and an **answer** field grouped under a **session_id**. Each account can have many session ids, i.e. we have a nested structure:

```
account_id
|__ session_id
|__ message (question or answer)
```

Most of the messages in the dataset are written in Russian. Languages were detected by fasttext-langdetect.

The markup was made in a combined way (Figure 4). We took our dataset and flagged as ‘suspicious’ sessions of those companies, whose reputation we weren’t certain of, because they aren’t our constant customers and aren’t well-known in the Russian business environment. Then we have manually evaluated messages inside sessions with ‘legal’ or ‘illegal’, so not the whole session was automatically marked as illegal.

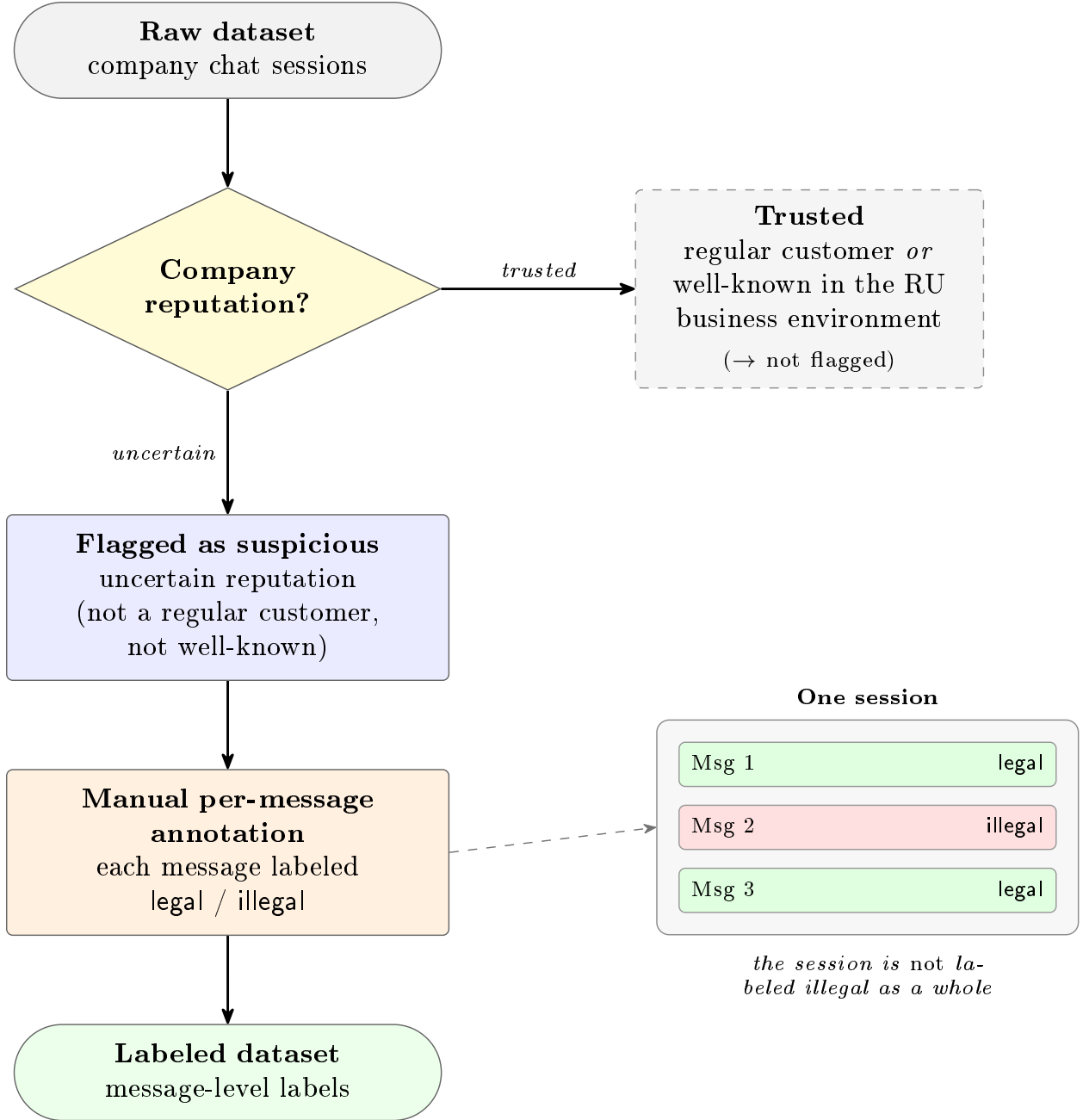


Figure 4: Data markup (annotation) pipeline: reputation-based flagging followed by manual per-message legal/illegal labeling. A session is not labeled illegal as a whole; labels are assigned at the message level.

The classes in our dataset are imbalanced. The distribution is as following:

Message label	Count	Percentage
legal	7816 116	99.99
illegal	463	0.01

Table 3: Class distribution in the full dataset.

However, the train dataset has to somehow be balanced. So we worked on a proper dataset separation.

All sessions containing at least one illegal message were identified (115 sessions) and extracted in full, yielding 1,156 messages (463 illegal + 693 legal). To approximate a 1:2 illegal:legal message ratio, the target number of legal messages was set to $2 \times 463 = 926$; since 693 are already present, an additional 233 were required. These were drawn from the pool of “clean” sessions (3,581,002 sessions containing no illegal message), which is randomly shuffled under a fixed seed (`random.seed(42)`); whole sessions were then added one at a time until the accumulated message count met or exceeded 233. This admitted 101 clean sessions contributing 236 messages (the slight overshoot above 233 occurred because sessions were added atomically rather than truncated). Concatenating the two pools produced a balanced dataset of 1,392 rows across 216 sessions (929 legal + 463 illegal).

The split was performed over the 216 *sessions* rather than over individual messages — a grouped partition. This was methodologically important: messages belonging to one conversational session were kept entirely within a single subset, preventing information leakage between training and evaluation data that would otherwise inflate performance estimates. A nominal 70/15/15 ratio was obtained through two nested applications of `train_test_split` (seed 42, no stratification): the first removed 15% of sessions as the test set; the second removed a fraction $0.15/0.85 \approx 17.6\%$ of the remaining 85% as the validation set, leaving $\sim 70\%$ for training. This produced 150 train / 33 validation / 33 test sessions. Messages were then assigned to a subset according to their session.

The final messages count in each sample is shown in Table 4.

Train	Validation	Test
748	336	305

Table 4: Number of messages in each split.

5 Experiments

5.1 Metrics

In this project the F1-score was used as a metric to evaluate the results of each model. F1-score is calculated as the harmonic mean of the Precision and Recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Here the Precision shows the number of true positive (TP) outcomes from the entire set of positive model answers ($TP + FP$), and the Recall shows the number of true positive (TP) outcomes from the entire set of positive actual samples ($TP + FN$):

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}.$$

For our project the recall value is more important than precision, because it’s better to send excess legal messages to moderators for a manual check-up than skip an illegal message.

However, F1-score only is not representative enough. Therefore we looked at the confusion matrix and classification report.

For a binary classification task, the confusion matrix is a 2×2 table that cross-tabulates the true class labels against the predicted class labels:

$$\begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix}.$$

Classification report was used to present a structured summary of the main evaluation metrics computed per class. For each class $c \in \{\text{legal}, \text{illegal}\}$, the report provides Precision, Recall, and F1-score as defined above, as well as the number of actual instances of that class in the evaluated set, referred to as *support*:

$$\text{support}(c) = TP_c + FN_c.$$

In addition to per-class metrics, the classification report includes two aggregate summaries.

Macro average computes the unweighted mean of each metric across all classes, treating each class equally regardless of its frequency:

$$\text{Macro } F_1 = \frac{1}{|C|} \sum_{c \in C} F_{1,c}.$$

Weighted average computes the mean of each metric weighted by the support of each class, thus reflecting the class imbalance present in the data:

$$\text{Weighted } F_1 = \frac{1}{N} \sum_{c \in C} \text{support}(c) \cdot F_{1,c},$$

where $N = \sum_{c \in C} \text{support}(c)$ is the total number of evaluated instances.

5.2 Experiment Setup

5.2.1 Rule-based method

The lexicon was assembled from a CSV dictionary. For each entry we collected both the normalized term and its spelling variants (stored as alternatives in a `regex_pattern` field and separated by the `|` delimiter), lower-cased and stripped them. The rule-based method was run once and evaluated directly on the held-out test split (305 messages). A message was flagged as *illegal* if a matching term appeared in either its `question` or its `answer` field.

We evaluated two matching strategies. The *naive* variant performs plain substring search, which inflates recall at the cost of many false positives. The *smart* variant constrains matches to whole-word boundaries via the regex pattern `\b<term>\b` and additionally flags weight-like expressions through the auxiliary pattern `\d+(?:\.\d+)?\s*[rr]` (a number optionally followed by a decimal part and a gram unit), which is intended to capture dosage/quantity mentions that the lexicon alone would miss.

5.2.2 LLMs

We wrote the first three prompt variants with the help of neural networks: “Minimalistic” (`prompt_a`) — a short prompt without a dictionary, containing only a general instruction; “Detailed” (`prompt_b`) — with a maximally detailed description, the full dictionary, and explicit rules; and “Compromise” (`prompt_c`) — balancing brevity and thorough description, and including a short version of the dictionary. The last two prompts reached a macro F_1 -score of 0.85.

As a result, we extended the instruction and obtained an improved prompt version — “Enhanced” (`prompt_d`). With it we achieved a recall (illegal) of 0.83 and a macro F_1 -score of 0.91. The final “Enhanced” (`prompt_d`) consists of detailed rules that define: (a) the positive class through several explicit conditions (direct mentions, slang and euphemisms, transliteration, action context, and emoji signals), together with positive-class triggers — entities related to unwanted topics, obfuscated channel and bot names, suspicious abbreviations and isolated Latin characters, references to cryptocurrency, and phrases with a double meaning; (b) the negative class (homonyms in everyday use, irony, irrelevant chat); (c) an explicit decision priority — context takes precedence over individual tokens; and (d) the full-size dictionary as a reference word list.

5.3 Baselines

In our project rule-based method was chosen as the first baseline, so the aim of further surveys and experiments was to beat a macro F1-score **0.68**.

6 Results

We have conducted various experiments which have shown the effectiveness of different methods and helped to decide which models to choose for further work.

First of all, we have tested two matching strategies of our rule-based baseline. The *naive* option has shown 1.0 recall for illegal messages but it was inflated at the cost of many false positives (Table 5). It means that it classified nearly every message as illegal, making it useless as a practical detector despite the perfect recall score. High recall is uninformative when the model cannot distinguish between the two classes.

	Precision	Recall	F_1 -score	Support
legal	1.00	0.09	0.16	202
illegal	0.36	1.00	0.53	103
Accuracy			0.40	305
Macro avg	0.68	0.54	0.35	305
Weighted avg	0.78	0.40	0.29	305

Table 5: Rule-based baseline, *naive* (substring) matching on the test split.

The *smart* option has shown better results (Table 6). Macro F_1 -score 0.68 is an acceptable value for this method, so it was chosen as a baseline score.

	Precision	Recall	F_1 -score	Support
legal	0.76	0.88	0.82	202
illegal	0.66	0.47	0.55	103
Accuracy			0.74	305
Macro avg	0.71	0.67	0.68	305
Weighted avg	0.73	0.74	0.72	305

Table 6: Rule-based baseline, *smart* (whole-word + weight-pattern) matching on the test split. The macro F_1 of 0.68 is used as the baseline target score.

Then we have tested partially LLM-generated prompts on different models using the train sample (Table 7). Here *Prompt* denotes the prompt variant (b / c) and whether the domain dictionary was embedded into it (*dict* / *no dict*); the precision, recall, and F_1 columns refer to the *illegal* class.

Model	Prompt	n	Accuracy	Precision	Recall	F_1
Qwen3.5-9B	b, no dict	748	0.8904	0.9927	0.6267	0.7684
	b, dict	748	0.8904	0.9927	0.6267	0.7684
	c, no dict	748	0.8890	0.9855	0.6267	0.7662
	c, dict	748	0.8904	0.9927	0.6267	0.7684
Gemma-4-E4B-it	b, no dict	748	0.8904	1.0000	0.6221	0.7670
	b, dict	748	0.8904	1.0000	0.6221	0.7670
	c, no dict	748	0.8904	1.0000	0.6221	0.7670
	c, dict	748	0.8904	0.9856	0.6313	0.7697
YandexGPT-5-Lite-8B-instruct	b, no dict	748	0.8730	0.7748	0.7926	0.7836
	b, dict	748	0.8369	0.7436	0.6682	0.7039
	c, no dict	748	0.8369	0.7684	0.6267	0.6904
	c, dict	748	0.8917	0.9928	0.6313	0.7718
Ministral-3-8B-Instruct-2512	b, no dict	716	0.8869	0.9762	0.6119	0.7523
	b, dict	693	0.8817	0.9692	0.6176	0.7545
	c, no dict	726	0.8636	0.8609	0.6250	0.7242
	c, dict	727	0.8776	0.9085	0.6293	0.7435

Table 7: Results on the train sample. Precision, recall, and F_1 are reported for the *illegal* class; n is the number of messages for which a verdict was returned.

6.1 Instruct versus Reasoning Models

In parallel with prompt development, we compared *instruct* models, which answer directly from the instruction, against *reasoning* models, which build an explicit chain of thought before producing the final answer, at the cost of extra time and tokens but with the potential for a more accurate result. For every model we report the configuration (prompt b or c, with or without dictionary) that produced the highest macro F_1 on the validation set (Table 8).

Type	Model	Best prompt	n	Legal			Illegal			Macro F_1
				P	R	F_1	P	R	F_1	
instruct	YandexGPT-5-Lite	prompt_b_no_dict	305	0.91	0.97	0.94	0.826	0.553	0.663	0.765
instruct	Gemma-4-E4B	prompt_c_with_dict	305	0.87	0.97	0.92	0.897	0.505	0.646	0.760
instruct	Qwen3.5-9B	prompt_c_with_dict	305	0.87	0.97	0.92	0.897	0.505	0.646	0.760
instruct	Ministral-3-8B	prompt_c_with_dict	300	0.87	0.96	0.91	0.839	0.510	0.634	0.750
reasoning	Qwen3.5-9B	prompt_c_with_dict	297	0.81	0.97	0.88	0.890	0.531	0.667	0.780
reasoning	Gemma-4-E4B	prompt_c_with_dict	305	0.80	0.97	0.88	0.900	0.515	0.654	0.760
reasoning	Ministral-3-8B	prompt_b_no_dict	305	0.79	0.97	0.87	0.900	0.505	0.646	0.760

Table 8: Best configuration per model, comparing instruct and reasoning modes on the validation set (prompt_b / prompt_c). n is the number of messages for which a verdict was returned; the best recall and macro F_1 within each mode are highlighted in bold.

Three conclusions follow from this comparison. First, YandexGPT-5-Lite-8B-instruct achieved the best recall on the illegal class (0.553) and the best macro F_1 (0.765) among instruct models, while the reasoning configuration of Qwen3.5-9B obtained the overall best recall (0.531) and macro F_1 (0.780) among reasoning models, but on fewer parsed items ($n = 297$ instead of 305). Second, reasoning mode did not outperform instruct mode by a meaningful margin for any model family, while consuming substantially more tokens and inference time. Third, both instruct and reasoning LLM configurations exceeded the smart keyword baseline (macro $F_1 = 0.68$).

We selected YandexGPT-5-Lite-8B-instruct for further work for three reasons: (i) as an instruct model it does not spend tokens on an explicit reasoning trace, which shortens inference time; (ii) it showed higher recall than the other candidates, which is critical for moderation, where missing a genuinely illegal message is more costly than over-flagging a legal one; (iii) Qwen3.5-9B failed to return a well-formed, parsable verdict for a fraction of the dataset rows, whereas YandexGPT consistently returned structured output.

We worked on the new prompt for YandexGPT-5-Lite-8B-instruct after the model’s mistake analysis and have measured scores on the test sample. The confusion matrix is shown in Table 9 and the corresponding classification report in Table 10.

	Predicted legal	Predicted illegal
True legal	194	7
True illegal	17	86

Table 9: Confusion matrix for YandexGPT-5-Lite-8B-instruct with the new prompt_d on the test split.

	Precision	Recall	F_1 -score	Support
legal	0.92	0.97	0.94	201
illegal	0.92	0.83	0.88	103
Accuracy			0.92	304
Macro avg	0.92	0.90	0.91	304
Weighted avg	0.92	0.92	0.92	304

Table 10: Classification report for YandexGPT-5-Lite-8B-instruct with the new `prompt_d` on the test split.

This configuration — YandexGPT-5-Lite-8B-instruct with `prompt_d` — was then carried forward as the base model for the in-context learning experiments described below.

6.2 In-Context Learning

6.2.1 ICL Baseline

To study the effect of demonstration choice, we started by supplying the model with $t = 10$ examples drawn from the validation set, under two sampling strategies — *random* (no class balancing) and *balanced* (1:1 stratified sampling of legal and illegal examples) — and with or without the full domain dictionary (`FULL_DICT`) embedded in the prompt. The baseline parameters were: model YandexGPT-5-Lite-8B-instruct, prompt `prompt_d` in its four variants, $t = 10$, evaluated on the validation split. Results are summarised in Table 11.

Version	Accuracy	Precision	Recall	Macro F_1
<code>prompt_d_random_few_shot_wo_dict</code>	0.5506	0.4826	0.7762	0.5451
<code>prompt_d_balanced_few_shot_wo_dict</code>	0.6310	0.5463	0.7832	0.6305
<code>prompt_d_random_few_shot_with_dict</code>	0.5744	0.5000	0.7692	0.5716
<code>prompt_d_balanced_few_shot_with_dict</code>	0.6399	0.5556	0.7692	0.6398

Table 11: ICL baseline on the validation set with the full dictionary (`FULL_DICT`), $t = 10$ examples. The best recall and macro F_1 are highlighted in bold.

The best recall was obtained with the balanced stratified sampling strategy without the dictionary embedded in the prompt (`prompt_d_balanced_few_shot_wo_dict`: recall = 0.7832, macro F_1 = 0.6398¹). Because the dictionary made no substantial difference to the outcome, we retained it in all subsequent experiments for consistency with the non-ICL pipeline.

6.2.2 Refining the ICL Baseline

We then added the number of few-shot examples t as a tunable hyperparameter, sweeping $t \in \{2, 4, 8, 16, 32\}$ for both sampling strategies and both dictionary settings. The best-performing configuration overall was the enhanced prompt with two randomly sampled examples (`prompt_d_random_with_dict_2_examples`, recall = 0.818). However, the best configuration did not show a decisive advantage over its closest competitors once the full set of metrics — not recall alone — was taken into account, so no single hyperparameter combination could be unambiguously declared optimal. We therefore carried five configurations forward for further analysis: `prompt_d_random_wo_dict_2_examples`, `prompt_d_balanced_wo_dict_2_examples`, `prompt_d_balanced_wo_dict_4_examples`, `prompt_d_random_with_dict_2_examples`, and `prompt_d_balanced_with_dict_2_examples`.

Table 12 shows the best results obtained across the hyperparameter sweep (all with $t = 2$), while Table 13 shows the weakest results, obtained with the largest example budget ($t = 32$); a large number of few-shot examples degraded accuracy and macro F_1 substantially, although recall remained comparatively high.

¹The corresponding macro F_1 in Table 11 is reported for the `with_dict` variant, which attained the best macro F_1 overall (0.6398), essentially tied with the `wo_dict` variant (0.6305).

Version	Accuracy	Precision	Recall	F_1	Macro F_1
prompt_d_balanced_with_dict_2_examples	0.899	0.943	0.811	0.872	0.894
prompt_d_balanced_wo_dict_2_examples	0.887	0.920	0.804	0.858	0.882
prompt_d_random_with_dict_2_examples	0.884	0.900	0.818	0.857	0.880
prompt_d_random_wo_dict_2_examples	0.875	0.898	0.797	0.844	0.870

Table 12: Best results of the ICL baseline with the example-count hyperparameter, $t = 2$. The best recall and macro F_1 are highlighted in bold.

Version	Accuracy	Precision	Recall	F_1	Macro F_1
prompt_d_random_wo_dict_32_examples	0.431	0.413	0.797	0.544	0.395
prompt_d_random_with_dict_32_examples	0.455	0.424	0.776	0.548	0.431
prompt_d_balanced_wo_dict_32_examples	0.482	0.440	0.797	0.567	0.461
prompt_d_balanced_with_dict_32_examples	0.500	0.449	0.776	0.569	0.487

Table 13: Worst results of the ICL baseline with the example-count hyperparameter, $t = 32$. Bold marks the extremes discussed in the text (lowest macro F_1 , highest recall within this group).

6.2.3 ICL with Semantic Embeddings

To further improve classification quality, we moved from arbitrary demonstration sampling to semantic selection: examples were chosen not at random but by their semantic proximity to the message being classified. Embeddings were generated with the Qwen3-8B embedding model, and for each test message the closest examples were retrieved from the labeled training pool by cosine similarity, so that the model’s context contained the demonstrations most relevant to that specific case rather than arbitrary ones.

Two strategies for injecting embeddings into the prompt were evaluated:

1. **Top- N semantic with class balancing.** The $N/2$ most similar legal and $N/2$ most similar illegal examples are selected from the training set by cosine similarity of embeddings. This guarantees that both classes are represented in the prompt, although some selected examples may be less relevant than in an unconstrained top- N selection.
2. **Class balancing with diversification (MMR-style selection).** $N/2$ legal and $N/2$ illegal examples are selected such that each is simultaneously similar to the test message and dissimilar to the examples already chosen, avoiding redundant demonstrations. At each step the example with the best trade-off between relevance and diversity is selected according to

$$\text{score} = \lambda \cdot \text{sim}(\text{test}) - (1 - \lambda) \cdot \text{sim}(\text{selected}),$$

where $\text{sim}(\text{test})$ is the cosine similarity to the test message, $\text{sim}(\text{selected})$ is the maximum cosine similarity to the already-selected examples, and λ balances relevance against diversity ($\lambda = 0.7$ was used as the main operating point).

Table 14 reports the results of the top- N semantic strategy with class balancing, and Table 15 reports the results of the class-balancing-with-diversification (MMR) strategy.

Version	Accuracy	Recall	Macro F_1
prompt_d_semantic_random_wo_dict_2expls	0.9082	0.8738	0.8979
prompt_d_semantic_balanced_wo_dict_2expls	0.5049	1.0000	0.4901
prompt_d_semantic_balanced_wo_dict_4expls	0.5213	0.9417	0.5149
prompt_d_semantic_random_with_dict_2expls	0.9148	0.8738	0.9047
prompt_d_semantic_balanced_with_dict_2expls	0.6426	0.9417	0.6426

Table 14: Top- N semantic selection with class balancing (validation set). The perfect recall obtained by `prompt_d_semantic_balanced_wo_dict_2expls` comes at the cost of a low macro F_1 (0.4901), i.e. the model over-predicted the illegal class; the best macro F_1 (0.9047) is obtained with two randomly-drawn semantic neighbours and the dictionary included.

#	Version	n	Accuracy	Precision	Recall	F_1 (binary) / Macro F_1
1	prompt_d_mmr_with_dict_2expls_lam70	305	0.7869	0.6218	0.9417	0.7490 / 0.7819
2	prompt_d_mmr_flat_with_dict_2expls_lam90	305	0.7869	0.6218	0.9417	0.7490 / 0.7819
3	prompt_d_mmr_flat_with_dict_2expls_lam50	305	0.7869	0.6218	0.9417	0.7490 / 0.7819

Table 15: Class balancing with diversification (MMR selection) on the test set, two examples per class, dictionary included. Results are stable across $\lambda \in \{0.50, 0.70, 0.90\}$; recall reaches 0.9417, the highest recall obtained in any experiment in this study.

The diversification (MMR) strategy delivered the best recall of the entire study (0.9417 on the test split, essentially matching the ≈ 0.94 figure obtained during validation), at the cost of a lower macro F_1 (0.78) than the plain `prompt_d` configuration without ICL (macro $F_1 = 0.91$, Table 10). Since recall on the illegal class is the metric of primary business importance for this project (Section 3, Metrics), MMR-based semantic diversification is the recommended configuration whenever maximising the detection of illegal content is prioritised over minimising the moderators’ false-positive workload; the plain `prompt_d` configuration remains preferable when a more balanced precision/recall trade-off is required.

7 Conclusion

This project addressed the problem of automatic detection of illegal drug trade in commercial chatbot logs, a task for which no established solution exists for the Russian language domain. Working with a real business dataset of 7.8 million messages, we constructed a labeled sample of 1,392 messages, designed a session-level train/validation/test split to prevent data leakage, and evaluated two families of methods: a rule-based keyword baseline and instruction-tuned LLMs with in-context learning.

As a first step, we assembled an 840-term bilingual lexicon of drug-related slang from four heterogeneous sources and implemented two matching strategies. The smart keyword baseline, combining whole-word boundary matching with a weight-pattern heuristic, reached a Macro F_1 of 0.68 on the test set and served as the target score for all subsequent experiments.

All tested LLMs — Qwen3.5-9B, Gemma-4-E4B-it, Ministral-3-8B-Instruct, and YandexGPT-5-Lite-8B-instruct — exceeded the baseline with Macro F_1 values between 0.73 and 0.78, and neither reasoning mode nor a larger parameter budget provided a decisive advantage over plain instruct inference. The best-performing model overall was YandexGPT-5-Lite-8B-instruct, which we selected for its combination of high recall, low latency, and reliably parsable output. After targeted prompt revision based on error analysis (`prompt_d`), it achieved a Macro F_1 of 0.91 on the test set, with recall of 0.83 on the illegal class and precision of 0.92 — a substantial improvement over the baseline and over all other tested configurations. This result demonstrates that lightweight, locally deployable LLMs can serve as effective content moderators for sensitive Russian-language business data without requiring fine-tuning or exposure of customer data to external services.

Building on this base model, we further optimised performance through in-context learning. A grid search over the number of demonstrations, the sampling strategy (random vs. 1:1 class-balanced), and the presence

of the domain dictionary showed that a small number of examples (two per class) generally outperformed larger few-shot budgets, and that balancing examples by class was consistently beneficial. Moving from arbitrary to semantically-selected demonstrations, and specifically to a strategy that balances relevance to the query with diversification among the selected examples (MMR-style selection, $\lambda = 0.7$), raised recall on the illegal class to 0.9417 on the test split — the highest recall observed in the study, though at a lower macro F_1 (0.78) than the non-ICL `prompt_d` configuration. This confirms that class balancing and within-class diversification of few-shot demonstrations are the key levers for maximising recall, which is the metric of greatest business importance for this project, since a missed illegal message is more costly than an extra manual review of a legal one.

References

- [1] Cascavilla, G.; Catolino, G.; Sangiovanni, M. Illicit Darkweb Classification via Natural-Language Processing: Classifying Illicit Content of Webpages based on Textual Information. In *Proceedings of the 19th International Conference on Security and Cryptography - Vol. 1: SECRYPT*, Lisbon, Portugal, 2022, pp. 620–626.
- [2] Li, J.; Xu, Q.; Shah, N.; Mackey, T. K. A Machine Learning Approach for the Detection and Characterization of Illicit Drug Dealers on Instagram: Model Evaluation Study. *J. Med. Internet Res*, 2019.
- [3] Prado-Sánchez, V.-P.; Domínguez-Díaz, A.; De-Marcos, L.; Martínez-Herráiz, J.-J. Zero-Shot Classification of Illicit Dark Web Content with Commercial LLMs: A Comparative Study on Accuracy, Human Consistency, and Inter-Model Agreement. *Electronics*, 2025.
- [4] Rupa, A. P.; Gangopadhyay, A. Multi-modal Deep Learning Based Fusion Approach to Detect Illicit Retail Networks from Social Media. *2020 International Conference on Computational Science and Computational Intelligence (CSCI)*, Las Vegas, NV, USA, 2020, pp. 238–243.
- [5] Shah, N.; Li, J.; Mackey, T. K. An Unsupervised Machine Learning Approach for the Detection and Characterization of Illicit Drug-dealing Comments and Interactions on Instagram. *Subst. Abus*, 2022.
- [6] Sonawane, M. S. et al. AI Detection of Drug Activity in Telegram Bots and Groups for Security Applications. *The 18th International Scientific Conference on Instrumentation Engineering*, 2025.
- [7] Song, M.; Jang, E.; Kim, J.; Shin, S. Covering Cracks in Content Moderation: Delexicalized Distant Supervision for Illicit Drug Jargon Detection. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*. Association for Computing Machinery, New York, NY, USA, 2025, pp. 1265–1276.
- [8] <https://www.kaggle.com/datasets/kirillkoncha/russian-illegal-drugs-reviews-sentiment-dataset?resource=download>
- [9] <https://pravorub.ru/personal/3340.html>
- [10] https://ru.wiktionary.org/wiki/%D0%9F%D1%80%D0%B8%D0%BB%D0%BE%D0%B6%D0%B5%D0%BD%D0%B8%D0%B5:%D0%A1%D0%BB%D0%B5%D0%BD%D0%B3_%D0%BD%D0%B0%D1%80%D0%BA%D0%BE%D0%BC%D0%B0%D0%BD%D0%BE%D0%B2
- [11] <https://github.com/minkyoo9/JEDIS>
- [12] https://github.com/s-nlp/inappropriate-sensitive-topics/blob/main/Version2/sensitive_topics/sensitive_topics.csv
- [13] <https://huggingface.co/apanc/russian-sensitive-topics>